

柔性设计的一个高频词是降低开发人员的精神负担，也就是少记一些概念，如果这些概念同属一个东西，那就应该将他们归类到一个地方，等等的一些技巧

这个世界物理上运行得非常有序的系统一定有它的内在逻辑，如果你搭建的模型不能满足他所有的运行规则，那么就需要回顾这个模型是否有所欠缺

[This is especially true in a process that emphasizes refactoring.](#)

[When software doesn't have a clean design, developers dread even looking at the existing mess, much less making a change that could aggravate the tangle or break something through an unforeseen dependency.](#)

当软件没有一个干净的设计时，开发人员甚至害怕看到现有的混乱，更不用说做出可能加剧混乱或打破某些东西的更改了

[In any but the smallest systems, this fragility places a ceiling on the richness of behavior it is feasible to build. It stops refactoring and iterative refinement.](#)

除了最小的系统之外，在任何系统中，这种脆弱性都为其可构建的功能的丰富程度设置了一个天花板。它停止重构和迭代细化。

[that从句](#)

[从句详解 > ^83b0df](#)

改成 that is feasible to build会更好理解一下

[Supple design is the complement to deep modeling.](#)

[Through the iterative cycle, you hammer that material into a useful shape, cultivating a model that simply and clearly captures the key concerns, and shaping a design that allows a client developer to really put that model to work.](#)

[To create elements that can be assembled into elaborate systems and still be understandable, a dedication to MODEL-DRIVEN DESIGN has to be joined with a moderately rigorous design style.](#)

[The same person might well play both roles—even switch back and forth in minutes—but the relationship to the code is different nonetheless.](#)

[It must follow the contours of a deep model of the domain, so most changes bend the design at flexible points.](#)

这里的意思是只要代码围绕着深度模型的轮廓展开，所有的变换都只是把设计扭到更加灵活的点

Intention-revealing interfaces

We are always fighting cognitive overload: If the client developer's mind is flooded with detail about how a component does its job, his mind isn't clear to work out the intricacies of the client design.我们总是在与认知过载作斗争:如果客户端开发人员的头脑中充斥着组件如何工作的细节, 他的头脑就不清楚客户端设计的复杂性

To obtain the value of explicitly modeling a concept in the form of a class or method, we must give these program elements names that reflect those concepts. The names of classes and methods are great opportunities for improving communication between developers, and for improving the abstraction of the system.

所以说, 起名字要起好

In the public interfaces of the domain: state relationships and rules, but not how they are enforced; describe events and actions, but not how they are carried out; formulate the equation but not the numerical method to solve it. Pose the question, but don't present the means by which the answer shall be found.

Entire subdomains can be carved off into separate modules and encapsulated behind INTENTION-REVEALING INTERFACES. Using such whittling to focus a project and manage the complexity of a large system will be discussed more in Chapter 15, "Distillation," with COHESIVE MECHANISMS and GENERIC SUBDOMAINS.

Side-Effect-Free Functions

In standard English, the term side effect implies an unintended consequence, but in computer science, it means any effect on the state of the system.

As soon as this arbitrarily deep nesting is involved, it becomes very hard to anticipate all the consequences of invoking an operation.

The use of the term side effect underlines the inevitability of that interaction.

The usefulness of any abstraction of interfaces is limited if the developers are forced to pierce the veil.

Obviously, you can't avoid commands in most software systems, but the problem can be mitigated in two ways.

But by definition, this segregation of side effects into simple command methods only applies to ENTITIES.

The side effect often can be completely eliminated by deriving a VALUE OBJECT instead of changing existing state, or by moving the entire responsibility into a VALUE OBJECT.

Strictly segregate commands (methods that result in modifications to observable state) into very simple operations that do not return domain information. Further control side effects by moving complex logic into VALUE OBJECTS when a concept fitting the responsibility presents itself.

严格隔离命令（导致对可观测状态修改的方法）到非常简单的不会返回领域信息的操作

Assertions

Separating complex computations into SIDE-EFFECT-FREE FUNCTIONS cuts the problem down to size, but there is still a residue of commands on the ENTITIES that produce side effects, and anyone using them must understand their consequences.

We need a way of understanding the meaning of a design element and the consequences of executing an operation without delving into its internals.

“design by contract”

约定的设计

Seek models with coherent sets of concepts, which lead a developer to infer the intended ASSERTIONS, accelerating the learning curve and reducing the risk of contradictory code.

Because ASSERTIONS are all in terms of states, rather than procedures, they make tests easy to write.

断言都是基于状态的

Theoretically, any noncontradictory set of assertions would work. But humans don't just compile predicates in their heads.

While developers were pondering this option, they made a discovery.

So, to make the volume model logically consistent would make it unsuitable for its application requirements. There seems to be a dilemma.

As we search for a better model, we have significant advantages over the original designers, because of the knowledge crunching and refactoring to deeper insight that has happened in the interim.

当我们寻找一个更好的模型时,我们在最初的设计师身上有很大的优势,因为知识的计算和重构是在临时发生的更深层次的洞察

The next ingredient in recombinable elements is effective decomposition. . . .

可重组元素的下一个成分是有效分解

Conceptual Contours

Sometimes people chop functionality fine to allow flexible combination. Sometimes they lump it large to encapsulate complexity.

When elements of a model or design are embedded in a monolithic construct, their functionality gets duplicated. The external interface doesn't say everything a client might care about. Their meaning is hard to understand, because different concepts are mixed together.

On the other hand, breaking down classes and methods can pointlessly complicate the client, forcing client objects to understand how tiny pieces fit together.

Half of a uranium atom is not uranium. And of course, it isn't just grain size that counts, but just where the grain runs.

But there is rhyme and reason somewhere, or else modeling would be pointless.

To avoid slipping into a mechanistic view of them, temper your technical thinking by frequently touching base with your intuition for the domain.

为了避免陷入对它们的机械论观点，你可以通过经常接触你对该领域的直觉来磨练你的技术思维。

slipping into

Is this an expedient based on a particular set of relationships in the current model and code, or does it echo some contour of the underlying domain?

Clumping things that don't need to be dissected or rearranged avoids clutter and makes it easier to see the elements that really are meant to recombine.

Decompose design elements (operations, interfaces, classes, and AGGREGATES) into cohesive units, taking into consideration your intuition of the important divisions in the domain.

将设计元素(操作、接口、类和aggregate)分解为内聚单元，将你对领域中重要划分的直觉考虑进去

it's hard to produce up front. But it may never emerge from technically oriented refactoring; it emerges from refactoring toward deeper insight.

这很难提前产生。但它可能永远不会出现在面向技术的重构中;它从重构走向更深入的洞察。

When successive refactoring tends to be localized, not shaking multiple broad concepts of the model, it is an indicator of model fit.

当连续的重构趋向于本地化，而不动摇模型的多个广泛概念时，它是模型拟合的一种提示。

The new model contained only one more object than the old one, yet the partitioning of responsibility had been greatly changed.

新模型只比旧模型多包含一个对象，但责任的划分却发生了很大的变化。

Schedules, which had been worked out through case logic in the Calculator classes, were exploded into discrete classes for different types of fees and interest. On the other hand, payments of fees and interest, previously kept separate, were lumped together.

在计算器类中通过案例逻辑计算出的时间表，被分解成不同类型的费用和利息的离散类。另一方面，以前分开的费用和利息的支付被合并在一起。

But had she found a CONCEPTUAL CONTOUR that will help the domain design change and grow as the application and the business evolve? There can be no guarantees about how a design will handle unanticipated change, but she thought it had improved the odds.

As the project proceeded, a requirement emerged for detailed rules for handling early and late payments.

Nor did it come because she made a design so versatile it could accommodate any conceivable change. It happened because in the previous refactoring, the design was aligned with underlying concepts of the domain.

INTENTION-REVEALING INTERFACES allow clients to present objects as units of meaning rather than just mechanisms. SIDE-EFFECT-FREE FUNCTIONS and ASSERTIONS make it safe to use those units and make complex combinations. The emergence of CONCEPTUAL CONTOURS stabilizes parts of the model and also makes the units more intuitive to use and combine.

Standalone Classes

And interdependencies pile up easily.

But even a MODULE can be a lot to think about without an almost fanatical commitment to controlling dependencies within it.

In every programming environment, a few basics are so pervasive that they are always in mind.
编程语言中有些语法是普遍的，人们都知道

Practically speaking, “integers” don’t add to the intellectual load.

Implicit concepts, recognized or unrecognized, count just as much as explicit references. Although we can generally ignore dependencies on primitive values such as integers and strings, we can’t ignore what they represent.

你认识string，但是你不一定记得它背后的含义，所以总是要把很多属性封装成一个可以理解的对象

This scrutiny starts with the factoring of the model concepts themselves. Then it requires attention to each individual association and operation. Model and design choices can chip away.

at dependencies—often to zero.

Every such self-contained class significantly eases the burden of understanding a MODULE.

Closure of Operation

Stripping interfaces down to deal with nothing but primitives can impoverish them.

Mathematicians are fanatical about not introducing extraneous concepts, and the property of closure provides them a way of defining an operation without involving any other concepts.

The property of closure tremendously simplifies the interpretation of an operation, and it is easy to think about chaining together or combining closed operations.

If the implementer has state that is used in the computation, then the implementer is effectively an argument of the operation, so the argument(s) and return value should be of the same type as the implementer.

如果实现者有会在计算中使用到的状态，那么实现者其实就是一个操作中的参数，所以参数和返回值就应该和实现者的类型一致

java中的lambda函数的来源

you can't just conjure up a new one to answer a question

The argument matches the implementer, but the return type is different, or the return type matches the receiver and the argument is different. These operations are not closed, but they do give some of the advantages of CLOSURE. When the extra type is a primitive or basic library class, it frees the mind almost as much as CLOSURE.

The usefulness of a MODEL-DRIVEN DESIGN is sensitive to the quality of the detailed design and implementation decisions, and it only takes a few confused developers to derail a project from the goal.

Declarative Design

ASSERTIONS can lead to much better designs, even with our relatively informal way of testing them.

This is tedious and fraught with error, and the bulk of it obscures the meaning of our model. (Some languages are better than others, but all require us to do a lot of grunt work.)

Generating a running program from a declaration of model properties is a kind of Holy Grail of MODEL-DRIVEN DESIGN, but it does have its pitfalls in practice.

The unintended consequence of many attempts at declarative design is the dumbing-down of the model and application, as developers, trapped by the limitations of the framework, enact design triage in order to get something delivered.

许多声明式设计尝试的意外结果是模型和应用程序的简化，因为开发人员被框架的限制所困，制定设计分类以获得交付的东西

This control code introduces side effects, so that the behavior is no longer dictated completely by the declared rules. Adding, removing, or reordering the rules can cause unexpected, incorrect results. Therefore, a logic programmer has to be careful to keep the effect of code obvious, just as an object programmer does.

The best of these unburden developers of drudge work while leaving them complete freedom to design.

Domain-Specific Languages

In this style, client code is written in a programming language tailored to a particular model of a particular domain.

I'm all in favor of learning advanced technology and design concepts, but we have to soberly assess the skills of a particular team, as well as the likely skills of future maintenance teams. Also, there is value in the seamlessness of an application and a model implemented in the same language.

我非常赞成学习先进的技术和设计理念，但我们必须冷静地评估特定团队的技能，以及未来维护团队可能的技能。同样，应用程序和用同一种语言实现的模型的无缝性也是有价值的。

Extending SPECIFICATION in a Declarative Style

SPECIFICATION is an adaptation of an established formalism, the predicate. Predicates have other useful properties that we can draw on, selectively.

However, other implementation options are possible that would minimize object count or boost speed, or perhaps be compatible with idiosyncratic technologies present in some project.

When you want to test a candidate, you have to interpret this structure, which can be done by popping off each element, then evaluating it or popping off the next as required by an operator. You would end up with this:

You have to find an implementation with trade-offs that work for your circumstances. The same pattern and model can underlie very different implementations.

But what they asked for was a list of chemicals whose handling has become more stringent. Perhaps there are none in-house right now, or perhaps they just happened to be packed into a more stringent container. In either case, the report just described would not list them.

In most situations it is best to avoid such complexity by making a choice, either forgoing some of the operators or forgoing subsumption.

This might be a point where the granularity is too coarse. The conventional approach would be to break the calculation methods down into subroutines.

Model concepts are decoupled; operations entangle a minimum of other types.

操作包含最少的其他类型